Web Application Security Assessment Report

Target: https://kitcoek.in/

Date: 10 February 2026
Prepared By: Adarsh Dipak Gurav
Prepared for: Future Interns
Project ID: PEN-2026-001

Confidentiality & Disclaimer

This document contains proprietary and confidential information. Duplication, redistribution, or use, in whole or in part, in any form, requires consent of both parties.

A penetration test is considered a snapshot in time. The findings and recommendations reflect the information gathered during the assessment (10-02-2026) and not any changes made outside of that period.

Contact Information

Role	Contact Details
Pentester	Name: Adarsh Dipak Gurav Email: adarshgurav1911@gmail.com Mobile: 8080914946

Executive Summary

On February 10, 2026, a security assessment was conducted on the web application https://kitcoek.in/. The assessment utilized both passive scanning and automated vulnerability scanning techniques.

The assessment identified vulnerabilities ranging from High to Informational severity. Most notably, the application is using an outdated JavaScript library with known critical CVEs, and lacks several standard security headers.

Methodology

The assessment followed the OWASP Top 10 standard for web application security testing.

• Scanning Mode: Automated and Passive Scanning.

• Approach: Identifying security gaps in HTTP headers, cross-domain configurations, and server responses.

Tools Used:

* Nmap (Network Mapper)
* OWASP ZAP (Zed Attack Proxy) - Passive & Automated Scan

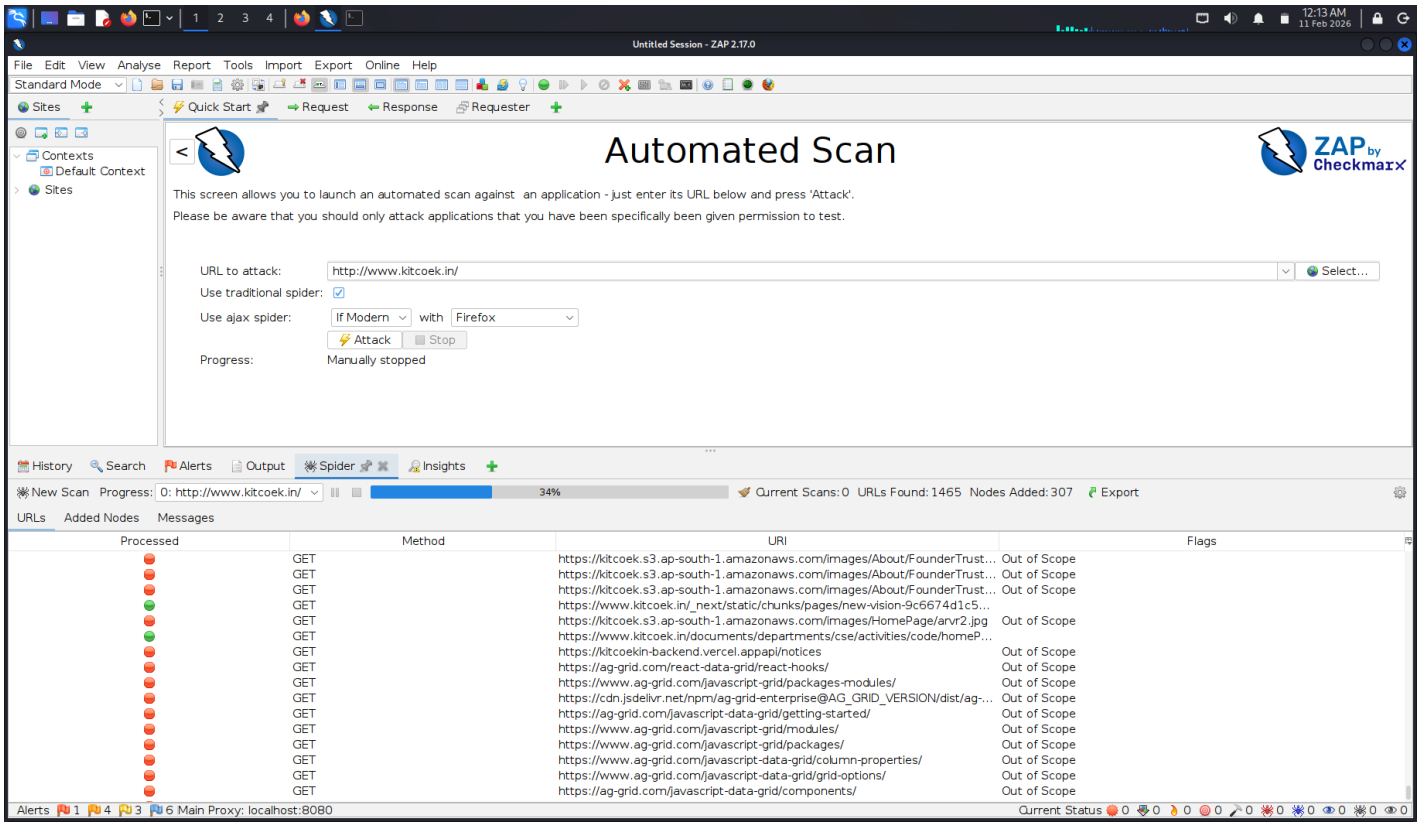


Figure 1 - Initial setup of the OWASP ZAP Automated Scan against the target application.

Findings Summary

The following alerts were generated during the automated scan:

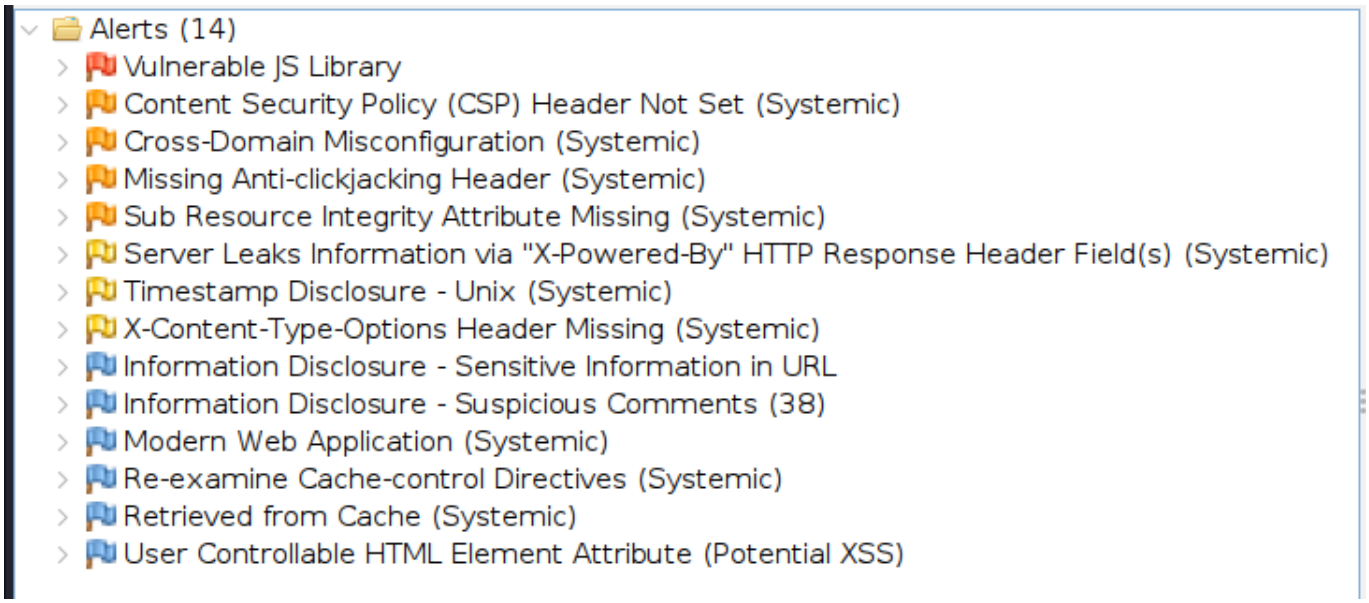


Figure 2 - Summary of all 9 security alerts identified by OWASP ZAP.

ID	Vulnerability Name	Severity	Recommendation
1	Vulnerable JS Library (Next.js)	High	Upgrade library to latest version
2	Cross-Domain Misconfiguration (CORS)	Medium	Restrict Access-Control-Allow-Origin
3	Missing Security Headers	Low	Implement X-Content-Type-Options & CSP
4	User Controllable HTML Attribute	Info	Validate/Sanitize input parameters

Detailed Penetration Test Findings

1) Vulnerable JS Library (Next.js)

Severity: High

URL: https://www.kitcoek.in/_next/static/chunks/main-3006aeb9b8182ed5.js

Description

The application is using a version of the Next.js library (v12.0.8) which is known to be vulnerable. Automated analysis identified references to CVE-2024-47831 and CVE-2025-57822.

Impact

Using components with known vulnerabilities can lead to a variety of attacks, including Cross-Site Scripting (XSS), data leakage, or remote code execution depending on the specific CVE.

Proof of Concept

Vulnerable JS Library

URL: https://www.kitcoek.in/_next/static/chunks/main-3006aeb9b8182ed5.js
Risk: High
Confidence: Medium
Parameter:
Attack:
Evidence: document.getElementById("__NEXT_DATA__").textContent);window.__NEXT_DATA__=D;t.version="12.0.8"
CWE ID: 1395
WASC ID:
Source: Passive (10003 - Vulnerable JS Library (Powered by Retire.js))
Input Vector:

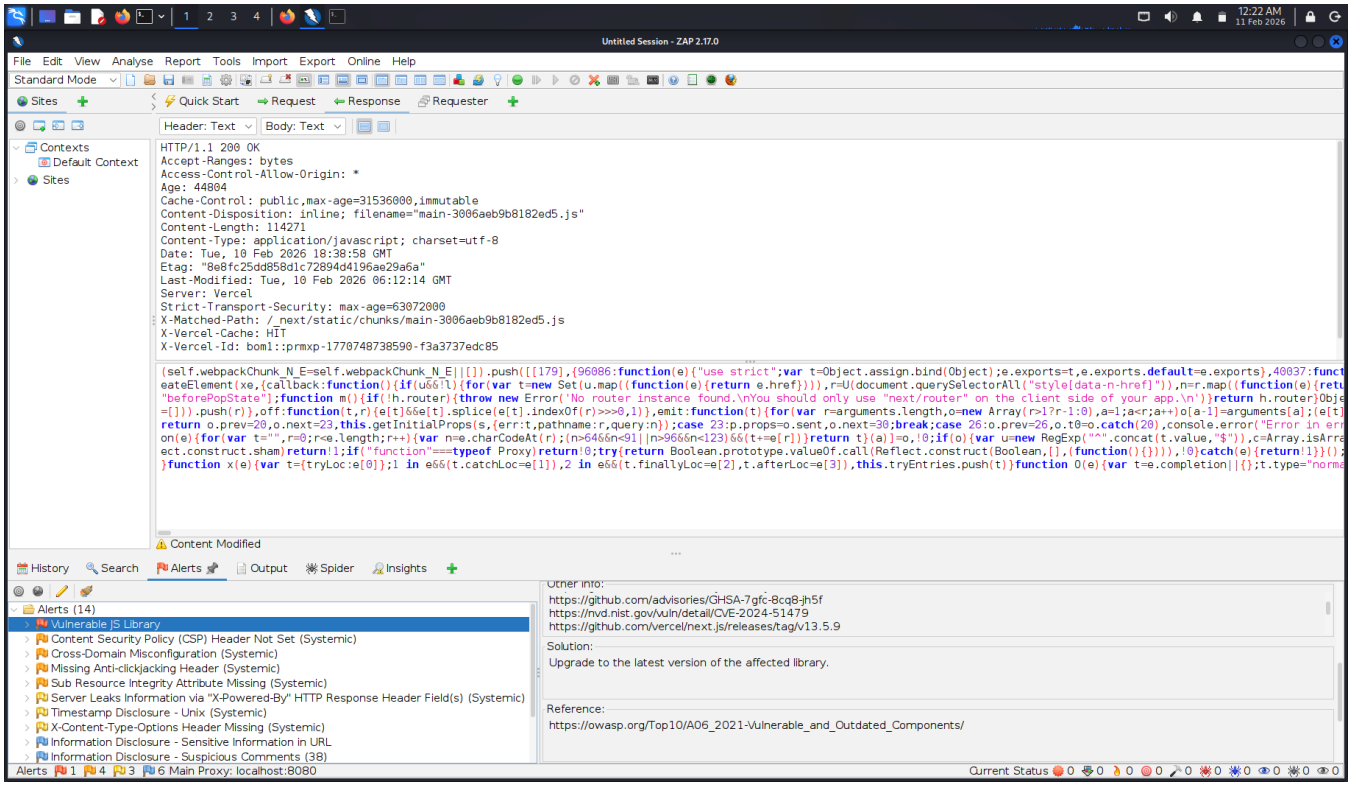
Description:
The identified library appears to be vulnerable.

Other Info:
The identified library nextjs, version 12.0.8 is vulnerable.
CVE-2024-47831
CVE-2025-57822

Solution:
Upgrade to the latest version of the affected library.

Reference:
https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/

Evidence: t.version="12.0.8" found in source code.



Recommendation

Upgrade Next.js to the latest stable version immediately.

2) Cross-Domain Misconfiguration

Severity: Medium

URL: https://www.kitcoek.in/robots.txt

Description

The web server is configured with an overly permissive Cross-Origin Resource Sharing (CORS) policy. The 'Access-Control-Allow-Origin' header is set to '*' (wildcard).

Impact

This allows any third-party domain to read the response from your application. While less critical on public files like robots.txt, if this configuration exists on sensitive API endpoints, it could allow attackers to steal user data.

Proof of Concept

Cross-Domain Misconfiguration

URL: https://www.kitcoek.in/robots.txt
Risk: Medium
Confidence: Medium
Parameter:
Attack:
Evidence: Access-Control-Allow-Origin: *
CWE ID: 264
WASC ID: 14
Source: Passive (10098 - Cross-Domain Misconfiguration)
Input Vector:

Description:

Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web server.

Other Info:

The CORS misconfiguration on the web server permits cross-domain read requests from arbitrary third party domains, using unauthenticated APIs on this domain. Web browser implementations do not permit arbitrary third parties to read the response from authenticated APIs, however. This reduces the risk somewhat. This misconfiguration could be used by an attacker to access data that is available in an unauthenticated manner, but which uses some other form of security, such as IP address white-listing.

Solution:

Ensure that sensitive data is not available in an unauthenticated manner (using IP address white-listing, for instance).
Configure the "Access-Control-Allow-Origin" HTTP header to a more restrictive set of domains, or remove all CORS headers entirely, to allow the web browser to enforce the Same Origin Policy (SOP) in a more restrictive manner.

Reference:

https://vulncat.fortify.com/en/detail?category=HTML5&subcategory=Overly%20Permissive%20CORS%20Policy

Recommendation

Configure the 'Access-Control-Allow-Origin' header to only allow specific, trusted domains, or remove the header entirely for public resources where it is not needed.

3) Missing Security Headers

Severity: Low

System: https://www.kitcoek.in/

Description

Several standard HTTP security headers are missing from the server response:

1. X-Content-Type-Options: nosniff (Prevents MIME-sniffing)
2. Content-Security-Policy (CSP) (Mitigates XSS and data injection)
3. Anti-clickjacking Header (X-Frame-Options)

Impact

Missing these headers increases the attack surface for Cross-Site Scripting (XSS), Clickjacking, and MIME-sniffing attacks.

Proof of Concept

X-Content-Type-Options Header Missing

URL: https://www.kitcoek.in/robots.txt
Risk: Low
Confidence: Medium
Parameter: x-content-type-options
Attack:
Evidence:
CWE ID: 693
WASC ID: 15
Source: Passive (10021 - X-Content-Type-Options Header Missing)
Input Vector:

Description:

The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.

Other Info:

This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type.
At "High" threshold this scan rule will not alert on client or server error responses.

Solution:

Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages.
If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.

Reference:

https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85)
https://owasp.org/www-community/Security_Headers

Recommendation

Configure the web server (Vercel/Next.js config) to send the following headers:

- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY or SAMEORIGIN
- Content-Security-Policy: [Define appropriate policy]

4) User Controllable HTML Attribute (Potential XSS)

Severity: Informational

URL: https://www.kitcoek.in/kit-gallery?Tab=2

Description

The value of the URL parameter 'Tab' appears to be reflected inside an HTML attribute in the response. ZAP identified this as a potential location for Cross-Site Scripting (XSS).

Impact

If the input is not properly sanitized, an attacker might be able to inject malicious JavaScript that executes in the context of the user's browser.

Proof of Concept

User Controllable HTML Element Attribute (Potential XSS)

URL: https://www.kitcoek.in/kit-gallery?Tab=2
Risk: Informational
Confidence: Low
Parameter: Tab
Attack:
Evidence:
CWE ID: 20
WASC ID: 20
Source: Passive (10031 - User Controllable HTML Element Attribute (Potential XSS))
Input Vector:

Description:
This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.

Other Info:
User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL:

https://www.kitcoek.in/kit-gallery?Tab=2

Solution:
Validate all input and sanitize output it before writing to any HTML attributes.

Reference:
https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

Recommendation

Ensure all user input is validated and output encoded before being rendered in the browser. Review the code handling the 'Tab' parameter.

Conclusion

Four security flaws with a severity ranging from High to Informational were found during the February 10, 2026, security assessment of https://kitcoek.in. The application's use of an antiquated Next.js library (High Severity) is the most important discovery; it exposes the program to known CVEs and needs to be patched right away.

In addition, configuration flaws that raise the risk of XSS and data theft were discovered, such as permissive CORS policies and missing HTTP security headers. It is highly advised to enforce more stringent server configurations and update the framework to the most recent stable version. Immediate attention to these problems will greatly improve the application's resistance to possible exploitation.